

CARBO

Guia do projeto para estudo no NotebookLM

Foco principal: script.js e a busca de alimentos

Material preparado a partir dos arquivos presentes no projeto em 6 de outubro de 2026. Os apêndices reproduzem os arquivos de texto atuais para que o NotebookLM possa consultá-los como fonte.

Como usar este guia

Este documento explica primeiro o comportamento da página principal e, com mais detalhe, cada parte de `script.js`. Depois, apresenta a estrutura do projeto e inclui os arquivos de texto completos como referência. A intenção é permitir perguntas como “o que dispara a busca?” e “por que a lista é apagada antes de procurar de novo?”.

Importante: o apêndice mantém os caracteres e o código exatamente como foram lidos dos arquivos atuais. Há trechos de texto com acentos corrompidos no projeto, por exemplo `vocêã`. Isso pode fazer as comparações da busca por nomes com acento não corresponderem ao que a pessoa digita sem acento.

O que o projeto faz hoje

A página principal mostra a marca Carbo e um campo de busca. Ao digitar, o JavaScript procura correspondências na lista local de alimentos e mostra sugestões clicáveis. A lista guarda o nome e a quantidade aproximada de carboidratos por 100 g.

O que ainda não faz

A versão atual não guarda uma refeição, não recebe gramas consumidas, não calcula carboidratos ingeridos ou calorias e não consulta uma IA, servidor ou banco de dados. Selecionar uma sugestão só preenche o campo e fecha a lista.

Mapa rápido do projeto

Arquivo/pasta	Responsabilidade
<code>index.html</code>	Estrutura da página, campo de busca, lista de sugestões e ordem de carregamento dos scripts.
<code>style.css</code>	Aparência: fundo, centralização, cores, campo, lista e rodapé.
<code>alimentos.js</code>	Array local com objetos de alimentos e valores aproximados de carboidratos por 100 g.
<code>script.js</code>	Escuta o que é digitado, filtra os nomes e monta sugestões clicáveis.
<code>aulas/</code>	Exercícios em subpastas por aula/vídeo.
<code>.vscode/</code>	Recomendações de extensões e configurações do editor.
<code>img/</code>	Logo, imagem de fundo e diagramas explicativos.

script.js explicado

O script usa os elementos HTML existentes e reage ao evento de digitação. O código abaixo é o script atual, preservado no apêndice integral.

```
const input = document.querySelector("#busca")
const sugestoes = document.querySelector("#sugestoes")

input.addEventListener("input", function () {
  sugestoes.innerHTML = ""

  if (input.value === "") return

  for (const alimento of alimentos) {
    if (alimento.nome.toLowerCase().includes(input.value.toLowerCase())) {
      const item = document.createElement("li")
      item.textContent = alimento.nome

      item.onclick = function () {
        input.value = alimento.nome
        sugestoes.innerHTML = ""
      }

      sugestoes.appendChild(item)
    }
  }
})
```

1. Encontrar os elementos HTML

```
const input = document.querySelector("#busca")
const sugestoes = document.querySelector("#sugestoes")
```

`document` representa a página carregada. `querySelector` procura nela um elemento que corresponda ao seletor CSS informado. O sinal `#` indica um id. Portanto, a primeira linha encontra o campo `id="busca"` e guarda sua referência em `input`; a segunda encontra a lista `id="sugestoes"` e guarda-a em `sugestoes`.

2. Esperar a digitação

```
input.addEventListener("input", function () {
```

`addEventListener` registra uma função para um evento. O evento `input` acontece quando o valor do campo muda: ao digitar, apagar ou colar texto. A função entre chaves roda a cada mudança. Esse nome de evento `input` e a variável `input` são coisas diferentes: um é o evento; o outro guarda o elemento HTML.

3. Limpar as sugestões antigas

```
sugestoes.innerHTML = ""
```

`innerHTML` representa o conteúdo interno do elemento. A string vazia `""` substitui o conteúdo atual por nada. Isso remove os itens sugeridos na busca anterior, para a lista ser reconstruída com o texto atual. Não apaga o campo de busca nem o array de alimentos.

4. Sair se o campo estiver vazio

```
if (input.value === "") return
```

`value` é o texto digitado no campo. `===` compara com igualdade estrita. Se o campo estiver vazio, `return` encerra esta execução da função. A lista já foi limpa na linha anterior; assim, não sobram sugestões

antigas.

script.js: busca e clique

5. Percorrer os alimentos

```
for (const alimento of alimentos) {
```

`alimentos` é o array definido em `alimentos.js`. O laço `for...of` visita seus itens um por vez. Em cada volta, a variável local `alimento` aponta para o objeto daquele alimento. `const` declara essa referência dentro do bloco do laço.

6. Conferir se o nome combina com o texto

```
if (alimento.nome.toLowerCase().includes(input.value.toLowerCase())) {
```

`alimento.nome` acessa a propriedade `nome` do objeto atual. `toLowerCase()` é um método de string que converte letras para minúsculas para comparar sem diferenciar maiúsculas de minúsculas.

`includes(...)` verifica se o nome contém o texto digitado. Essa busca não remove acentos: `feijao` não necessariamente encontra `feijão`.

7. Criar um item de sugestão

```
const item = document.createElement("li")
item.textContent = alimento.nome
```

`createElement("li")` cria um elemento HTML `<li>` novo, inicialmente fora da página. `textContent` define o texto simples que aparecerá nele, com o nome do alimento.

8. Definir o que acontece ao clicar

```
item.onclick = function () {
  input.value = alimento.nome
  sugestoes.innerHTML = ""
}
```

`onclick` recebe uma função que será executada quando a pessoa clicar nesse item. A primeira linha de dentro preenche o campo com o nome escolhido. A segunda limpa a lista de opções. Ela deixa o texto selecionado no campo; não salva o item numa refeição.

9. Colocar o item na lista visível

```
sugestoes.appendChild(item)
```

`appendChild` anexa o elemento `item` como filho da lista `sugestoes`. A partir daí, o navegador pode mostrá-lo. Essa linha mantém cada sugestão visível enquanto o laço continua; ao clicar, o manipulador apaga a lista com `innerHTML = ""`.

Fluxo de exemplo

Se a pessoa digita `arr`: o navegador dispara o evento; o script apaga as opções anteriores; percorre o array; compara `arr` com cada nome em minúsculas; cria um `<li>` para cada correspondência e anexa à lista. Ao clicar numa opção, o nome completo vai para o campo e a lista é limpa.

Ordem dos scripts no HTML

```
<script src="./alimentos.js"></script>
<script src="./script.js"></script>
```

O HTML carrega primeiro o array `alimentos` e depois o código que o consulta. Essa ordem permite que `script.js` encontre a variável quando a busca for usada.

Dados e conceitos relacionados

Formato dos dados

```
const alimentos = [  
  { nome: "Arroz branco cozido", carboidratosPor100g: 28.1 },  
  { nome: "Feijão carioca cozido", carboidratosPor100g: 13.6 },  
]
```

`alimentos` é o nome da constante; o valor atribuído é um array [...]. Cada objeto {...} representa um alimento. `nome` e `carboidratosPor100g` são propriedades. O texto entre aspas é um valor do tipo string; o número decimal é um valor do tipo number. O nome não é uma constante separada: é uma propriedade dentro de cada objeto.

Variáveis e escopo que aparecem no script

`input` e `sugestoes` são referências guardadas com `const`; o elemento HTML em si continua podendo mudar suas propriedades, como `value`. `alimento` representa cada objeto durante uma volta do laço. `item` é uma nova variável local para cada correspondência e guarda o elemento de lista criado.

Vocabulário curto

Termo	Significado neste projeto
Constante (const)	Declaração que não permite reatribuir aquele identificador.
Propriedade	Dado nomeado dentro de um objeto, acessado com ponto, como <code>alimento.nome</code> .
Método	Função associada a um valor, chamada com parênteses, como <code>nome.toLowerCase()</code> .
Evento	Acontecimento que o navegador informa ao código, como <code>input</code> ou <code>clique</code> .
Callback	Função fornecida para ser executada depois, por exemplo ao ocorrer o evento.
DOM	Representação em objetos dos elementos HTML que o JavaScript pode localizar e alterar.

Dados aproximados

A descrição de `alimentos.js` informa valores aproximados de carboidratos por 100 g e referência à TACO/NEPA-UNICAMP. O arquivo contém 30 entradas, incluindo um nome duplicado na versão atual. O projeto não calcula calorias. Valores alimentares podem variar conforme alimento, preparo e fonte; a lista é material didático local.

Exercícios das aulas

Os exemplos separados praticam caixas de diálogo do navegador, conversão para string, concatenação, template string, método `toUpperCase()` e incremento de número. No exercício que usa `String(prompt(...))`, somar as duas respostas com `+` concatena textos; para soma numérica seria necessário converter os dados para número.

Como abrir o projeto

Abra `index.html` no navegador ou use a extensão Live Server do VS Code. A busca é executada localmente no navegador com os arquivos JavaScript incluídos no HTML. Não há instalação de dependências de servidor necessária para essa demonstração.

Arquivos visuais e configurações

Imagens presentes em `img/`

`carbo-logo.png` - Logo exibida no cabeçalho da página principal.

`back.png` - Imagem de fundo referenciada no CSS.

`explicacao-alimentos.svg` - Diagrama sobre array, constante, propriedades e valores.

`explicacao-script-busca.svg` - Diagrama explicativo do fluxo de busca no script.

Extensões recomendadas pelo workspace

Prettier, Material Icon Theme, Omni Theme e Live Server. O README explica como instalá-las usando `code --install-extension`. As configurações em `.vscode/settings.json` configuram o editor, formatação, tema, ícones e perfil Git Bash para o terminal integrado.

Estrutura das aulas

`aula04/ex001.html` demonstra `alert`, `confirm` e `prompt`. `aula06/ex002.html` usa `prompt` para obter nome e mostra uma saudação. `ex003.html` recebe duas respostas e concatena strings. `ex004.html` acessa `toUpperCase()` e template string. `aula07/ex.js` incrementa 3 para 4 e usa `console.log`.

Apêndice: arquivos de texto completos

Os blocos abaixo preservam o conteúdo textual dos arquivos atuais para consulta. Arquivos de imagem são descritos no inventário anterior e não são reproduzidos como código.

README.md

Carbo

Projeto de estudos e exercícios de programação. A página principal apresenta a identidade visual do Carbo, e as pastas de aula guardam exemplos feitos durante as aulas.

Tecnologias usadas

- **HTML** para estruturar as páginas.
- **CSS** para os estilos da página principal e dos exercícios.
- **JavaScript** para interações e exercícios com `window.alert`, `window.confirm`, `window.prompt` e template strings.
- **Git** e **GitHub** para versionar o projeto. Repositório: [gabx2/carbo](https://github.com/gabx2/carbo).

Estrutura

- `index.html` – página principal do Carbo.
- `style.css` – estilos da página principal.
- `alimentos.js` – lista local de 30 alimentos com carboidratos aproximados por 100 g, referenciados à TACO/NEPA-UNICAMP.
- `img/` – imagens usadas pelo projeto.
- `aulas/` – exercícios organizados por aula/vídeo:
 - `aula04/` – `ex001.html`.
 - `aula06/` – `ex002.html`, `ex003.html` e `ex004.html`.
 - `aula07/` – `ex.js`.
- `.vscode/` – recomendações de extensões e configurações do VS Code.

Editor e extensões

O projeto contém estas extensões recomendadas para o Visual Studio Code, definidas em `.vscode/extensions.json`:

- **Prettier - Code formatter** (`esbenp.prettier-vscode`) – formatação do código.
- **Material Icon Theme** (`PKief.material-icon-theme`) – ícones de arquivos e pastas.
- **Omni Theme** (`rocketseat.theme-omni`) – tema de cores do editor.
- **Live Server** (`ritwickdey.LiveServer`) – servidor local para visualizar páginas durante o desenvolvimento.

As configurações do workspace ativam a formatação ao salvar e colar, usando o Prettier como formatador padrão, com aspas duplas, indentação de 2 espaços e sem ponto e vírgula. O tema configurado é Omni, e os ícones são do Material Icon Theme.

Instalar em outro computador

Instale o **Git** e o **Visual Studio Code**. Depois, clone o repositório e abra a pasta:

```
``bash
git clone https://github.com/gabx2/carbo.git
cd carbo
code .
``
```

Ao abrir o projeto, o VS Code pode sugerir as extensões recomendadas. Também é possível instalá-las pelo terminal integrado com estes comandos (o comando `code` precisa estar disponível no terminal):

```
``bash
code --install-extension esbenp.prettier-vscode
code --install-extension PKief.material-icon-theme
code --install-extension rocketseat.theme-omni
code --install-extension ritwickdey.LiveServer
``
```

As configurações do workspace ficam em `.vscode/settings.json` e são aplicadas ao abrir a pasta no VS Code.

Terminal e sugestões de comandos

No PowerShell, o recurso de sugestões do terminal se chama **Predictive IntelliSense** e é fornecido pelo módulo **PSReadLine**. O perfil do PowerShell deste ambiente configura as sugestões em modo `ListView`, que mostra opções enquanto você digita.

Para ativar essas sugestões no PowerShell de outro computador, execute:

```
``powershell
Install-Module -Name PSReadLine -Scope CurrentUser -Force
Set-PSReadLineOption -PredictionSource History
Set-PSReadLineOption -PredictionViewStyle ListView
``
```

O PSReadLine oferece o recurso de sugestões; o histórico local do PowerShell é usado como fonte das sugestões.

No VS Code, o terminal integrado está configurado para abrir com **Git Bash** no Windows (`.vscode/settings.json`).

Abrir o projeto

Abra `index.html` no navegador. Se a extensão Live Server estiver instalada, também é possível clicar com o botão direito no arquivo e escolher **Open with Live Server**.

index.html

```
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Carbo</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Lato:wght@400;700&display=swap"
      rel="stylesheet"
    />
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <header>
      <a href="index.html">
        
      </a>
      <div class="campo-busca">
        <input
          type="text"
          id="busca"
          placeholder="O que você comeu hoje?"
          autocomplete="off"
        />
        <ul id="sugestoes"></ul>
      </div>
    </header>
    <footer>
      <a
        href="https://github.com/gabx2"
        target="_blank"
        rel="noopener noreferrer"
        aria-label="GitHub"
      >
        <svg viewBox="0 0 16 16" aria-hidden="true">
          <path
            d="M8 0C3.58 0 0 3.58 0 8c0 3.54 2.29 6.53 5.47 7.59.4.07.55-.17.55-.38 0-.19-.01-.82-.01-1.49-2
            .01-3.7-2.53-.49-2.69-.94-.09-.23-.48-.94-.82-1.13-.28-.15-.68-.52-.01-.53.63-.01 1.08.58 1.23.82.72 1.21 1.8
            7.87 2.33.66.07-.52.28-.87.51-1.07-1.78-.2-3.64-.89-3.64-3.95 0-.87.31-1.59.82-2.15-.08-.2-.36-1.02.08-2.12
            0 0 .67-.21 2.2.82.64-.18 1.32-.27 2-.27.68 0 1.36.09 2 .27 1.53-1.04 2.2-.82 2.2-.82.44 1.1.16 1.92.08 2.12
            .51.56.82 1.27.82 2.15 0 3.07-1.87 3.75-3.65 3.95.29.25.54.73.54 1.48 0 1.07-.01 1.93-.01 2.2 0 .21.15.46.55
            .38A8.013 8.013 0 0016 8c0-4.42-3.58-8-8-8z"
          />
        </svg>
      </a>
    </footer>
    <script src="./alimentos.js"></script>
    <script src="./script.js"></script>
  </body>
</html>
```

style.css

```
* {
  box-sizing: border-box;
}

body {
  background-color: #171717;
  margin: 0;
  min-height: 100vh;
  display: flex;
  justify-content: center; /* centraliza na horizontal */
  align-items: center; /* centraliza na vertical */
  background: url(./img/back.png) no-repeat center center/cover;
}

header {
  text-align: center;
  padding: 10px;
}

header img {
  display: block;
  margin: auto;
  width: 300px;
  max-width: 100%;
  height: auto;
  margin-bottom: 20px;
}

#busca {
  font-family: "Lato", sans-serif;
  width: 100%;
  background-color: #fbf6e5;
  color: #171717;
  max-width: 300px;
  padding: 12px 16px;
  font-size: 14px;
  border: 1px solid #171717;
  border-radius: 8px;
  outline-color: #171717;
  text-align: center;
}

.campo-busca {
  position: relative;
}

#sugestoes {
  position: absolute;
  top: calc(100% + 4px);
  left: 0;
  right: 0;
  margin: 0;
  padding: 4px 0;
  list-style: none;
  background-color: #fbf6e5;
  color: #171717;
  border: 1px solid #171717;
  border-radius: 8px;
  font-family: "Lato", sans-serif;
  font-size: 14px;
  text-align: center;
  max-height: 240px;
  overflow-y: auto;
}

#sugestoes:empty {
  display: none;
}

#sugestoes li {
  padding: 8px 16px;
  cursor: pointer;
}

#sugestoes li:hover {
```

```
    background-color: #f3ecd4;
  }

  footer {
    position: fixed;
    bottom: 16px;
    left: 50%;
    transform: translateX(-50%);
  }

  footer a {
    display: block;
    color: #fbf6e5;
    padding: 8px;
    border-radius: 50%;
    transition: background-color 0.2s;
  }

  footer a:hover,
  footer a:focus-visible {
    background-color: rgba(251, 246, 229, 0.15);
  }

  footer svg {
    display: block;
    width: 24px;
    height: 24px;
    fill: currentColor;
  }

  h1 {
    border: 3px solid black;
  }
  p {
    color: #fbf6e5;
  }
```

alimentos.js

```
const alimentos = [  
  { nome: "Arroz branco cozido", carboidratosPor100g: 28.1 },  
  { nome: "Arroz integral cozido", carboidratosPor100g: 25.8 },  
  { nome: "Aveia em flocos crua", carboidratosPor100g: 66.6 },  
  { nome: "Biscoito maisena", carboidratosPor100g: 75.2 },  
  { nome: "Biscoito cream cracker", carboidratosPor100g: 68.7 },  
  { nome: "Farinha de trigo", carboidratosPor100g: 75.1 },  
  { nome: "Pão francês", carboidratosPor100g: 58.6 },  
  { nome: "Pão integral", carboidratosPor100g: 49.9 },  
  { nome: "Macarrão instantâneo", carboidratosPor100g: 62.4 },  
  { nome: "Milho verde cru", carboidratosPor100g: 28.6 },  
  { nome: "Milho verde enlatado", carboidratosPor100g: 17.1 },  
  { nome: "Mandioca cozida", carboidratosPor100g: 30.1 },  
  { nome: "Batata inglesa cozida", carboidratosPor100g: 11.9 },  
  { nome: "Batata-doce cozida", carboidratosPor100g: 18.4 },  
  { nome: "Inhame cru", carboidratosPor100g: 23.2 },  
  { nome: "Polvilho doce", carboidratosPor100g: 86.8 },  
  { nome: "Cuscuz de milho cozido", carboidratosPor100g: 25.3 },  
  { nome: "Feijão carioca cozido", carboidratosPor100g: 13.6 },  
  { nome: "Feijão preto cozido", carboidratosPor100g: 14.0 },  
  { nome: "Lentilha cozida", carboidratosPor100g: 16.3 },  
  { nome: "Grão-de-bico cozido", carboidratosPor100g: 27.4 },  
  { nome: "Banana-prata crua", carboidratosPor100g: 26.0 },  
  { nome: "Banana-nanica crua", carboidratosPor100g: 23.8 },  
  { nome: "Banana-maçã crua", carboidratosPor100g: 22.3 },  
  { nome: "Maçã Fuji crua", carboidratosPor100g: 15.2 },  
  { nome: "Abacaxi cru", carboidratosPor100g: 12.3 },  
  { nome: "Laranja-pera crua", carboidratosPor100g: 8.9 },  
  { nome: "Goiaba vermelha crua", carboidratosPor100g: 13.0 },  
  { nome: "Tomate cru", carboidratosPor100g: 3.1 },  
  { nome: "Inhame cru", carboidratosPor100g: 23.2 },  
]
```

script.js

```
const input = document.querySelector("#busca")
const sugestoes = document.querySelector("#sugestoes")

input.addEventListener("input", function () {
  sugestoes.innerHTML = ""

  if (input.value === "") return

  for (const alimento of alimentos) {
    if (alimento.nome.toLowerCase().includes(input.value.toLowerCase())) {
      const item = document.createElement("li")
      item.textContent = alimento.nome

      item.onclick = function () {
        input.value = alimento.nome
        sugestoes.innerHTML = ""
      }

      sugestoes.appendChild(item)
    }
  }
})
```

.vscode/extensions.json

```
{
  "recommendations": [
    "esbenp.prettier-vscode",
    "PKief.material-icon-theme",
    "rocketseat.theme-omni",
    "ritwickdey.LiveServer"
  ]
}
```

.vscode/settings.json

```
{
  // editor
  "editor.wordWrap": "on",
  "editor.fontSize": 18,
  "editor.lineHeight": 30,
  "editor.tabSize": 2,
  "editor.bracketPairColorization.enabled": true,
  "editor.guides.bracketPairs": true,
  "editor.minimap.enabled": false,
  "editor.formatOnSave": true,
  "editor.formatOnPaste": true,

  // explorer
  "explorer.compactFolders": false,
  // workbench
  "workbench.editor.enablePreview": false,
  "workbench.iconTheme": "material-icon-theme",
  "workbench.colorTheme": "Omni",

  // prettier
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "prettier.enable": true,
  "prettier.singleQuote": false,
  "prettier.tabWidth": 2,
  "prettier.semi": false,

  // terminal
  "terminal.integrated.fontSize": 16,
  "terminal.integrated.profiles.windows": {
    "Git Bash": {
      "source": "Git Bash"
    }
  },
  "terminal.integrated.defaultProfile.windows": "Git Bash"
}
```

aulas/aula04/ex001.html

```
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>FIRST</title>
    <h1>FIRST</h1>
    <style></style>
  </head>
  <body>
    <script>
      window.alert("Olá, seja bem-vindo(a)!")
      window.confirm("Você deseja continuar?")
      window.prompt("Qual é o seu nome?")
    </script>
  </body>
</html>
```

aulas/aula06/ex002.html

```
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>FIRST</title>
    <h1>FIRST</h1>
    <style></style>
  </head>
  <body>
    <script>
      var nome = window.prompt("Qual é o seu nome?")
      window.alert("Olá, " + nome + "! Seja bem-vindo(a)!")
    </script>
  </body>
</html>
```

aulas/aula06/ex003.html

```
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>FIRST</title>
    <h1>FIRST</h1>
    <style></style>
  </head>
  <body>
    <script>
      var n1 = String(window.prompt("Digite um numero:"))
      var n2 = String(window.prompt("Digite outro numero:"))
      var soma = n1 + n2
      window.alert("A soma dos dois numeros é: " + soma)
    </script>
  </body>
</html>
```

aulas/aula06/ex004.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
    <style>
      body {
        font: normal 20px arial;
      }
    </style>
  </head>
  <body>
    <script>
      var nome = window.prompt("Qual é o seu nome?")
      var idade = window.prompt("Qual é a sua idade?")
      document.write(`Olá, ${nome.toUpperCase()}! Você tem ${idade} anos.`)
    </script>
  </body>
</html>
```

aulas/aula07/ex.js

```
var n = 3  
n = n + 1  
console.log(n)
```